Efficiency Profiling of Small Language Models on Edge Hardware

1. Introduction

Adam Dalwai, Mahean Seam, and Vrushti Patel | Topic 1 | 13 November 2026

Research report, Mac phase; Android phase pending. This report documents an experiment designed to measure small language model inference under the memory, compute, and thermal limits of edge hardware. The selected models are Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct, each represented by official Q4_K_M and Q8_0 GGUF files. The user approved this pair after the original proposal selected TinyLlama 1.1B and Qwen2.5 3B. The historical proposal remains part of the record; the executable configuration identifies the adopted experiment.

The project asks three operational questions. First, how do model scale and weight quantization change prefill time, decode throughput, memory use, and observed power on an Apple M1 MacBook Air? Second, how does populated context change these costs when input length rises from 256 to 2,048 tokens? Third, can a pinned CPU testbed produce sufficiently comparable evidence for a later Android replication? These questions require separate timing boundaries because a fast token generator can still feel slow if prompt processing or loading dominates.

The two-device study is intentionally phased. The Mac work provides an executable baseline and reusable protocol. A teammate will later run Android experiments using the handoff and return raw evidence. The phone's exact model, chipset, OS, RAM, sensors, and available process memory must be recorded then. An estimated 6-8 GB of phone RAM guided model selection; it is not a measured specification or proof of runtime feasibility. This document therefore cannot support a laptop-versus-phone performance conclusion until those measurements exist.

The contribution is a reproducible measurement chain: pinned model files and runtime source, exact token workloads, per-run logs, monitoring records, coverage-aware analysis, and evidence-aware artifacts. The user's current presentation scope is one Mac observation per configuration; this exploratory sample does not estimate variability or repeatability. The full controlled repeated protocol remains documented for later completion. This is an experimental systems contribution rather than a claim of a new quantization algorithm. The public repository is https://github.com/addiee13/project_effAI. Local changes and newly generated artifacts require a separate publication check before the public repository can be claimed to contain them.

Reading guide. Sections 2-8 establish background, design, implementation, and measurement validity. Sections 9-10 present the available empirical evidence and interpretation. Sections 11-12 cover implications, limits, reproducibility, and the path to completing the two-device study. References follow the substantive report pages. Every numerical performance statement must resolve to retained raw observations; missing experiments remain visible.

2. Related Work and Technical Background

Small language models reduce storage and arithmetic relative to larger models, but parameter count alone does not determine edge-device cost. A checkpoint also fixes architecture, attention design, tokenizer, vocabulary, and training. The Qwen2.5 technical report establishes the model family and instruction-tuned checkpoint lineage [1]. Its published benchmark claims concern the authors' evaluations, not this project's M1 or Android measurements. We use the model family as the workload under test and measure local behavior directly.

The official Qwen GGUF repositories provide matching Q4_K_M and Q8_0 artifacts for the two selected checkpoints [2,3]. These labels describe concrete GGUF representations; they should not be interpreted as every tensor using exactly four or eight bits. Within each checkpoint, comparing the two supplied variants controls model identity while changing the weight representation. Comparing 0.5B and 1.5B keeps the family fixed but still changes architectural dimensions. It is therefore stronger than a cross-family deployment comparison for controlling lineage, while remaining insufficient to isolate a universal effect of parameter count.

llama.cpp supplies a common implementation for CPU inference and platform-specific builds [4]. Its llama-bench tool supports prompt processing and generation microbenchmarks and machine-readable output. Its benchmark documentation explicitly states that tokenization and sampling are excluded [5]. This distinction motivates our use of an instrumented real-prompt request for client-observed first-token and inter-token timing. Microbenchmark throughput and application response time answer different questions; neither should silently replace the other.

KV caching reuses past attention keys and values during autoregressive generation. Weight quantization changes the representation of learned tensors; KV-cache quantization changes runtime state. A focused F16-versus-Q8_0 cache comparison therefore complements the main weight comparison, with one model and long context fixed. Any performance difference also reflects backend and kernel support. Cache allocation, occupied context, and observed resident memory must be reported separately.

Quality is a separate axis. The proposal specifies perplexity on WikiText-2. The WikiText dataset was introduced as a language-model evaluation corpus with longer text dependencies [6]. Perplexity evaluates likelihood under a fixed tokenization and context policy; it does not establish instruction-following quality. A future evaluation should compare quantization deltas within a model on the same held-out text and settings, preserve the corpus checksum, and avoid claiming that lower perplexity proves a better assistant. WikiText-2 was not evaluated in the presentation pilot.

The present study differs from model-card capability evaluation by measuring phase-specific resource cost on a constrained local machine. Its main validity challenge is measurement control: background work, cached prompts, thermal drift, sampling boundaries, and unavailable sensors can change what a number means. Those engineering choices are central to the methodology, rather than incidental setup details.

3. Methodology: Factors, Controls, and Scope

The adopted experiment uses a CPU baseline, one request at a time, four threads, and a pinned llama.cpp source revision. The main matrix contains 2 model sizes x 2 weight variants x 3 input lengths x 2 output lengths, or 24 configurations per device and 48 across both devices. The two output budgets are 128 and 512 tokens, resolving the assignment brief's additional 512-output condition that was absent from the original proposal. A complete matrix is counted from successful, validated measurements rather than from scheduled configurations.

The full controlled protocol schedules one warm-up and three measured repetitions per configuration. Warm-ups are retained but excluded from headline means. A fixed random seed defines the order, and the actual executed order and timestamps remain evidence. At least 120 seconds separate repetitions of the same configuration. A strict thermal gate additionally requires a stable temperature window; elapsed time alone does not establish stabilization. For the current presentation pilot, the user authorized one measured Mac request per configuration instead. That 24-request exploratory session has n=1 for each point, so it reports no sample SD, significance, or repeatability estimate. Failures, timeouts, early stops, and unsupported cases are preserved rather than removed to make coverage appear complete.

The main matrix fixes the KV cache at F16. The planned focused study uses Qwen2.5-1.5B-Instruct Q4_K_M at 2,048 input and 128 output tokens, comparing F16 and Q8_0 cache. This would separate the weight and cache questions. Scheduler/concurrency scaling is outside this baseline; parallel requests remain one. A separate operator-profile run may explain where time is spent, but its instrumented timings would not enter the headline table. KV, quality, monitoring-overhead, and operator-profile studies were not run in the presentation pilot.

The runtime source is pinned to commit 79967ec596c0dacfd2251b085a57e79df292b1cc (tag b6784 in the study configuration). The Mac build uses Release mode, Metal disabled, Accelerate enabled, and zero GPU layers. This is a CPU baseline with platform numerical libraries, not a claim that the M1's CPU and an Android CPU have identical software kernels. Build options, binary checksum, startup backend logs, thread count, and batch settings jointly identify the execution environment.

The design narrows causal claims. A Q4/Q8 difference within a checkpoint can be attributed to that file/runtime combination under the controls, not to all four-bit quantizers. A cross-device difference later includes processor, OS, memory system, and platform implementation. Matching token counts standardizes a workload dimension but does not prove equal semantic work. A model's minimum useful quality is not inferred from speed.

The revised pair addresses two earlier feasibility concerns. TinyLlama's researched 2,048-token context could not fit a 2,048-token formatted input plus generation without changing the workload or validating context extension. The originally proposed 3B Q8 file raised runtime-memory risk for the estimated phone. These were planning concerns, not observed device failures. The smaller pair is user-approved; Android feasibility still requires a successful largest-condition pilot.

4. Methodology: Exact Workloads and Repetition

Input lengths refer to actual tokens after formatting, not to words or a configured context limit. A reproducible workload starts from a versioned text corpus, applies the declared request formatting, tokenizes with the selected checkpoint, and saves the exact token array used by the runtime. The expected input count must agree with the request and runtime timing record. This avoids accidental comparison of nominally equal character lengths that contain different token counts.

The configured context capacity is 3,072 positions. The largest planned occupied context is 2,048 input plus 512 output tokens, leaving headroom within that capacity. Allocation can reserve cache space for the configured capacity even when fewer positions are occupied. Consequently, the analysis distinguishes capacity, initial prompt length, generated tokens, and final occupied context. Decode measurements are collected after the requested input has been processed, so they reflect generation at the intended populated context.

Generation uses temperature zero and a declared seed. The benchmark forces the output budget through an ignore-EOS setting so that variable early stopping does not silently alter the timing workload. This produces a controlled throughput workload rather than an ordinary conversational completion policy. Logs retain output counts and stop reasons. Separate demonstration text should disclose this policy; a forced 512-token continuation is not evidence that a user needed or received 512 useful tokens.

Prompt caching is disabled for timed requests. Reusing an already evaluated prefix could make long-prompt prefill appear cheaper without doing the intended work. The warm-up controls first-use effects but does not authorize measured prompt reuse. The runtime's reported prompt tokens and cache behavior must be checked against the intended request. Server startup/loading is recorded separately from request timing; a steady-state request should not be called a cold-start measurement.

Repetition provides information about variability rather than merely increasing the sample count. Each aggregate preserves three individual observations, the arithmetic mean, and sample standard deviation with denominator n-1. The run order is randomized under a saved seed to reduce confounding between configuration and time. Long cooldowns and stable temperature reduce drift but cannot remove all DVFS or background-process variation. The evidence records time, power conditions, thermal state, and monitoring availability so that anomalies can be interpreted.

Before the full controlled sweep, the largest condition must complete with the expected input/output counts, no context overflow, the intended backend, and usable monitoring. The user approved a single-pass presentation pilot, so its first validated request demonstrates the recorded path and exact counts but does not establish the strict thermal gate, long-run repeatability, or Android memory headroom. The pilot's 30-second bounded temperature observation is exploratory; its 40.85-50.25 C reference window is not described as stable. Any change to batch size, cache type, backend, workload, or prompt construction after a frozen session creates a new configuration identity and is documented rather than blended into the old group.

5. System Implementation and Evidence Flow

The testbed separates configuration, execution, raw evidence, analysis, and presentation. Machine-readable study configuration specifies the factors and controls; a model manifest supplies repository revisions, filenames, byte sizes, and SHA-256 checksums. Large model files and local runtime builds live outside Git. Retrieval scripts reconstruct the local cache while verifying file identity. This reduces repository size while retaining a precise description of what was tested.

The runtime builder retrieves the pinned source, records platform build options, and creates CPU binaries. A successful build is only the first validation stage. The server must start, report its backend, load the intended model, accept the tokenized request, and return the complete response. A source-archive build can report an unknown binary version because it lacks Git metadata; the source revision, build records, and executable checksum are then stronger evidence than the version string alone. This issue must remain explicit in the environment record.

The request driver uses a local streaming endpoint and a monotonic clock. It records the request start, first response token, subsequent token events, final event, and runtime timing fields. Raw stream events are retained so that client-visible metrics can be recomputed. A server event can contain more than one token or transport buffering; observed event intervals are not automatically per-token kernel latency. Runtime token counts and request-side timestamps are interpreted together, with metric names preserving their boundary.

Monitoring runs beside inference and preserves timestamped samples. Mac process memory, CPU temperature when available, thermal-pressure state, utilization, charging/power state, and component-power readings have separate units and sources. Missing fields remain null or unavailable. A null GPU temperature does not become zero degrees. CPU-only execution does not make system background GPU activity impossible, so monitoring is contextual evidence rather than a substitute for backend logs.

A run identifier joins configuration, startup logs, request payload/token counts, generated output, timing, memory, monitoring, errors, and completion state. Analysis consumes saved records and emits coverage tables, aggregates, and plots. Figures must cite the data snapshot and grouping policy. The report and deck derive numerical tables from that same snapshot, which reduces the chance that a manually copied number drifts from the retained evidence.

The public testbed link is https://github.com/addiee13/project_effAI. At artifact-build time, a local isolated branch may contain work that is not yet published at that URL. Publication and repository access are final checks, not assumptions. The handoff must include the exact setup and run commands, dependencies, expected directory structure, and analysis invocation. A teammate should be able to reproduce a small validated case before committing hours to the full Android sweep.

6. Measurement: Latency, Throughput, and Memory

The loading boundary begins when a new model process is launched and ends when the runtime is ready. End-to-end request latency begins immediately before sending the request and ends when its response completes. Time to first token begins at the same request start and ends at the first delivered token event. These intervals differ: loading may happen before the request, while first-token latency includes request handling and prompt processing. Reporting them separately prevents a warm server's responsiveness from being presented as cold-start performance.

Runtime prefill time measures prompt evaluation under the runtime's definition. Runtime decode throughput divides generated tokens by the runtime's decode interval. The request-side wall-clock interval includes additional overhead such as serialization, event delivery, and sampling. Both are useful, but their denominators are different. A plot must identify which quantity it uses. The first-token metric is not calculated by dividing prompt tokens by decode speed, and microbenchmark timings are not substituted for application timing.

For a generated sequence with N tokens, an inter-token estimate requires a declared policy. Event-to-event intervals can describe streaming responsiveness, while a runtime average can describe compute cost. If the first token is already counted in the prompt-evaluation stage or an event carries multiple tokens, naive averaging produces a biased result. Retain event timestamps and actual output count, report the chosen definition, and avoid implying that an average captures tail latency. Very short smoke outputs are insufficient for estimating stable decode behavior.

Memory has several valid boundaries. A GGUF file's byte size is storage, not process peak RSS. Resident memory can include weights, allocated KV cache, work buffers, code, and runtime overhead; operating-system compression, file mapping, and shared pages complicate interpretation. The analysis uses the platform's declared process metric, converts bytes to MiB using 2^20, and labels sampled peaks as sampled rather than guaranteed instantaneous maxima. macOS time-command maximum resident set size, if used, needs its documented unit conversion.

Fixed context capacity helps control reserved KV allocation across input-length conditions but also means that an input-length increase need not raise allocated-cache memory proportionally. A memory chart therefore includes the capacity and cache type in its caption. The planned focused cache study asks whether Q8_0 changes observed memory and cost under the same capacity; it does not assume a theoretical twofold reduction in total RSS. This sub-study was not run. Weight-size ratios likewise do not establish memory-footprint ratios.

Three repeats support mean and sample standard deviation for each configuration. They provide limited statistical power, and overlapping variability should temper rankings. Missing and failed runs remain visible in coverage counts. An aggregate with n=1 is a pilot observation, not a three-run result. Cross-device aggregation is deferred until Android logs use compatible workload definitions and explicitly labeled platform memory metrics.

7. Measurement: Power, Energy, and Thermal Control

Energy measurements are meaningful only with a stated boundary. The saved macmon samples include estimated CPU, GPU, and ANE power. For 22 of the 24 requests, the samples bracketed the client request interval with adequate cadence for a rough gross component-rail integration. The result is request-boundary estimated component energy, not whole-device energy and not decode-only energy. Screen, storage, peripherals, charging, and unreported components are outside this estimate. A separate powermetrics log supplies estimated component samples and system thermal pressure; it is supplementary evidence, not a wall meter.

Power in milliwatts is divided by 1,000 to obtain watts. For timestamped samples, energy over an interval is approximated by integrating power against elapsed seconds, for example with the trapezoidal rule. Sampling cadence, gaps, and boundary overlap affect that estimate. Decode joules per token is valid only when decode energy is isolated over the same decode phase and divided by the actual generated-token count. Whole-request or background-inclusive energy divided by tokens must be labeled accordingly rather than called isolated decode energy.

The Mac was drawing from AC and charging at the session inventory capture (86% battery); per-run power state is retained. The operator enabled a caffeinate sleep-prevention assertion at 05:52:10 UTC, after 11 requests had begun. This changed the power-management context during the exploratory session and is a validity limitation. CPU-temperature samples during inference ranged from 74.7 to 92.9 C, while 1,071 powermetrics thermal-pressure samples were nominal. These observations do not establish that temperature was stable or that no throttling occurred. GPU temperature was unavailable. The Android plan may use an inline USB power meter; its boundary would differ from Mac component rails, so cross-device energy rankings require comparable measurement boundaries and charging policy.

An idle baseline can help distinguish workload-associated power from the platform's background draw, but baseline subtraction is an analytical policy rather than an automatic correction. Record when idle was sampled, how it was matched to power state, and whether reported energy is gross or baseline-subtracted. Negative or implausible deltas should trigger investigation rather than clipping. No energy-efficiency claim should be made from a temperature reading, battery percentage change over a short run, or nominal processor TDP.

Thermal stabilization uses CPU temperature sampled every ten seconds. The configured gate requires a sixty-second window whose range is at most one degree Celsius and whose values are no more than two degrees above the recorded idle reference. The timeout is 1,800 seconds. The implementation and run evidence must confirm that the rule was enforced. Runs without usable temperature are exploratory under the study configuration, even if a fixed cooldown elapsed.

Temperature, thermal pressure, clocks/utilization, and throughput drift jointly inform throttling analysis. A warm device alone does not prove throttling; a falling throughput alone can reflect background interference. The M1 MacBook Air's behavior is interpreted from observed sensors and logs. Android sensor availability and units require validation on the actual phone. Power-adapter state, battery level, screen state, background work, and ambient conditions should be documented consistently enough to explain changes in the evidence.

8. Validation, Overhead, and Failure Handling

The testbed has successive validation gates. Retrieval verifies model-file identity. Build validation verifies executable behavior, source provenance, and backend. A small request verifies the stream/parser and schema. The largest-condition pilot verifies context feasibility and full output counts. A repeat verifies that the same configuration can complete again under the cooling rule. These gates are distinct: installation is not a successful experiment, a four-token smoke test is not a full pilot, and one pilot is not a completed matrix.

Monitoring may perturb the workload. A controlled overhead check compares matched requests with and without the selected monitoring stack, using the same model, token counts, backend, power state, and repeat policy. Individual observations and order are retained. The relative difference is computed against the unmonitored baseline. With few pairs, a small difference cannot be confidently distinguished from ordinary run variation, so the report states the evidence and uncertainty. Instrumented operator profiling is kept outside headline timing even when its overhead appears small.

Operator-level claims require a separate evidence path. Runtime or system-profiler traces can identify expensive kernels or sampled call stacks, but the interpretation must match the profiler's granularity. A sample percentage is not automatically an exact fraction of elapsed inference time; a CPU profile cannot establish NPU utilization. Prefill-versus-decode timings support phase-level bottleneck analysis even when a detailed kernel attribution is unavailable. The study reports that limit rather than manufacturing a top-k operator chart.

The planned quality evaluation has its own validity checks. The same held-out WikiText-2 bytes, split, tokenization, context size, and chunk policy must be used within each model's Q4/Q8 comparison. The evaluation log should retain the number of tokens/chunks and final perplexity. WikiText-2 was not run in the current presentation scope, and no accuracy or instruction-following claim is inferred from the efficiency measurements.

Failure records include the attempted configuration, timestamp, command or request, exit/error status, partial output count, monitoring state, and reason for exclusion. A thermal timeout is not a successful cool run. A process kill may indicate memory pressure but requires supporting evidence before it is called an out-of-memory failure. A missing power sampler is not a measured zero. Retrying a configuration creates another attempt rather than erasing the first record.

Artifact validation also has concrete gates: report page count and visual inspection, slide legibility and speaker timing, video duration and decoded playback, link correctness, and traceability from figures to raw evidence. These checks establish usable local deliverables. They do not establish remote publication, Android completion, instructor acceptance, or iCollege submission. The final status ledger preserves those distinctions so that the presentation's narrative remains consistent with the evidence.

9. Empirical Results and Analysis: Evidence Coverage

The exploratory Mac CPU presentation sweep completed all 24 unique main configurations. An independent audit checked prompt/request token identity, exact runtime input/output and SSE counts, limit stop reason, no truncation, no prompt-cache reuse, and runtime throughput denominators; all 24 passed. There is one observation per configuration (n=1), so these are descriptive points with no SD or repeatability estimate. Across unlike settings, runtime decode throughput ranged 26.34-83.66 tokens/s, request wall time 1.97-34.55 s, and sampled request-window process RSS 588-2131 MiB. Android remains pending.

Coverage is an empirical result about the testbed, not a forecast. The current Mac presentation matrix has 24 configurations and 24 validated measured requests, one per configuration. The full controlled two-device matrix has 48 configurations; with three repetitions it would require 144 measured requests plus warm-ups. The current sample is exploratory and does not meet that repeated-study target. Focused cache, quality, operator, and monitoring-overhead experiments remain follow-up work; none should inflate the main-matrix coverage count.

A coverage audit groups records by device, model identity, weight quantization, input count, output budget, cache type, and backend. Only validated successful measured records enter descriptive comparisons. Mean and sample SD are reported only for groups with repeated observations. Warm-up, exploratory, interrupted, failed, and wrong-token-count attempts remain in the evidence inventory. The current groups have n=1 and fail the full three-repetition coverage target.

The table and plots attached to the final data snapshot must preserve the intended comparison. For weight quantization, compare Q4_K_M and Q8_0 within a checkpoint at equal input/output counts. For input length, hold model, weight type, cache, output count, and backend fixed. For the 128-versus-512 generation question, identify both total request cost and per-token decode behavior, since a longer output necessarily increases work and may warm the device during generation.

Android is pending teammate execution. Its empty coverage is not plotted as zero latency, zero energy, or missing bars with an implied rank. The cross-device results will be added only after hardware identification, largest-case pilot, backend validation, and compatible raw records. At this stage the strongest justified claims concern the observed Mac requests; device-general claims remain hypotheses.

The results manifest is the bridge between this document and the logs. It identifies included run paths, source snapshot checksums, aggregation rules, and generated figure files. Rebuilding the report should refresh its numerical content from this manifest rather than preserve manually typed values. Any disagreement between the rendered table and raw records is an artifact bug that must be fixed before submission.

10. Empirical Results and Analysis: Interpreting Comparisons

At the largest 2,048-input/512-output case, the four observed model/quantization points were: qwen05_q4 Q4_K_M: decode 60.15 tokens/s, request 15.21 s, sampled RSS 590 MiB; qwen05_q8 Q8_0: decode 61.42 tokens/s, request 14.87 s, sampled RSS 765 MiB; qwen15_q4 Q4_K_M: decode 30.40 tokens/s, request 34.55 s, sampled RSS 1181 MiB; qwen15_q8 Q8_0: decode 27.26 tokens/s, request 33.31 s, sampled RSS 2028 MiB. These are individual observations, not a model ranking. Across the 24 requests, CPU-temperature samples during requests ranged 74.7-92.9 C; GPU temperature was unavailable. A separate powermetrics log recorded nominal system thermal pressure in 1,071 samples, but temperature stabilization under the strict rule was not demonstrated. macmon component-power samples bracketed 22/24 request intervals sufficiently for gross component-rail integration; these are request-boundary estimates, not isolated decode energy or whole-device energy. Native caffeinate was enabled at 05:52:10 UTC during the sweep; 11/24 requests began before that power-management change. KV cache, WikiText-2 perplexity, monitoring-overhead, and operator-profiling studies were not run in this presentation scope.

A faster Q4 variant would be consistent with reduced weight traffic or a more favorable implementation, but it would not by itself prove a memory-bandwidth bottleneck. A speedup can reflect quantization kernels, arithmetic, cache behavior, or scheduling. Operator evidence and controlled phase comparisons are needed to strengthen the explanation. Conversely, a smaller file can fail to improve latency if dequantization overhead or another phase dominates.

Longer prompts increase the amount of prefill work. Their influence on decode must be evaluated at the actual populated context, with the fixed capacity declared. A context knee requires enough measured points and uncertainty to establish a change in behavior; three input lengths may reveal a trend but do not locate a precise architectural threshold. Extrapolation to maximum advertised context is outside the measured range.

Output-budget comparisons distinguish responsiveness from sustained generation. First-token time may be similar for equal prompts, while a 512-token run reveals a longer thermal and decode trajectory. The report should examine throughput and event intervals over time where timestamps permit it, rather than treating every token as identically costly. A single mean can hide warm-up, transport batching, or frequency changes during a request.

Memory and power charts are interpreted under their boundaries. Lower sampled RSS does not necessarily mean lower physical device energy. Estimated component power cannot be compared directly with USB input power. A low-precision KV cache can reduce allocated state while changing kernel cost or numerical behavior; only the focused matched study supports that local tradeoff. Missing sensors limit attribution but do not invalidate independently collected latency measurements.

The current one-observation-per-configuration sample does not estimate variability. The observed points show that both selected models completed all tested Mac requests under the pinned CPU build, and that runtime decode speed, request time, and sampled process RSS varied across the fixed workload dimensions. These are testbed observations, not evidence of a causal memory bottleneck or universal Q4/Q8 ordering. The report states the exact model, quantization, context, power, and thermal context and identifies the confounds. Repeated runs remain necessary for a controlled recommendation.

11. Implications, Insights, and Threats to Validity

The first deployment implication is methodological: choose a model using runtime evidence for the largest intended workload, not only a GGUF size or parameter count. Available RAM, cache capacity, buffers, operating-system pressure, and per-process limits all matter. The selected 0.5B/1.5B pair reduces planning risk for a 6-8 GB phone, but only the later Android pilot can validate that choice. A feasible Mac configuration is not proof of phone feasibility.

The second implication is that the preferred quantization depends on the outcome being optimized. Lower storage or sampled memory may favor Q4_K_M, while quality or a particular kernel implementation may favor Q8_0. A recommendation should state the task's minimum quality, latency target, memory headroom, and power boundary. This study does not collapse these objectives into an unexplained single efficiency score. Any Pareto tradeoff is limited to the measured variants and settings.

Internal validity depends on workload and thermal control. Cached prefixes, unrecorded power transitions, background renders, changing thread counts, and token-count mistakes can produce convincing but incomparable numbers. Exact token arrays, saved order, and disabled prompt reuse reduce some risks. The exploratory session did not demonstrate strict temperature stabilization, and a sleep-prevention assertion changed during the run. Those conditions are retained as limitations; one observation per setting provides no estimate of run-to-run variation.

Construct validity concerns what each metric represents. Runtime decode throughput measures a defined execution interval; streaming event latency measures delivery; RSS measures a process boundary; component rails measure a power boundary; perplexity measures likelihood on a corpus. Calling all of these user experience, device energy, or answer quality would overstate what was observed. Clear metric names and denominators preserve useful evidence without pretending that it answers every deployment question.

External validity is limited by one Mac, one eventual phone, two related checkpoints, and one runtime revision. A CPU baseline aids replication but does not characterize Android NPUs or the M1 Metal backend. Quantization behavior may differ for other architectures, newer runtime kernels, concurrent users, different languages, or longer contexts. The family-controlled pair also cannot answer the assignment's broad model-family question independently; that scope limit is documented rather than hidden.

The project journal supplies the engineering narrative: model research exposed a context mismatch and phone-memory risk; existing installed binaries required operational validation; instrumentation boundaries required a direct runtime path; source and artifact dependencies required reproducible fallbacks. These are useful lessons when supported by files and logs. The presentation should select the decisions that explain the final methodology, retain unsuccessful attempts honestly, and avoid inventing elapsed effort or portraying every routine step as a major obstacle.

12. Conclusion and Reproducibility Plan

This project develops a reproducible CPU profiling testbed for Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct at Q4_K_M and Q8_0. It separates model loading, prompt processing, token generation, client timing, process memory, thermal state, and estimated component power. The 256/1,024/2,048 input conditions and 128/512 output conditions cover the assignment's sequence-length dimensions. The current Mac presentation pilot completed all 24 configurations once, with no sample SD or repeatability estimate. The focused KV-cache comparison and separate quality, overhead, and operator runs are follow-up work, not current results.

The evidence shows the pinned Mac CPU testbed completed each of the 24 exact-token configurations with the requested output count and no truncation or prompt-cache reuse. It supports descriptive observations for this Mac, model set, CPU build, and run context only. One sample per configuration cannot establish repeatability or statistical significance. Thermal stabilization was not demonstrated; CPU-temperature observations ranged widely and a power-management assertion changed during the sweep. No Android result exists, so no two-device comparison or phone-feasibility claim is supported. The full repeated protocol and Android handoff remain available for later completion.

Reproduction begins with the public repository, https://github.com/addiee13/project_effAI, and the exact experiment revision. Run the model retrieval script to obtain the four checksum-verified files outside Git, build the pinned runtime with recorded options, and perform a small request/schema check. Then validate the largest workload, power conditions, monitoring, and the thermal gate. Execute the saved randomized schedule only after these checks. Rebuild aggregates and figures from the raw evidence before rebuilding the report, slides, and demo.

For the Android phase, a teammate records phone identity and system state, builds the same runtime source for the CPU baseline, verifies model checksums, and runs the smallest smoke case followed by the largest-condition pilot. Sensor availability, thermal units, memory metrics, charging policy, and energy boundary must be validated on the actual hardware. Unsupported sensors and configurations remain explicitly unavailable. Returning the complete raw-results directory, hardware record, commands, and failure ledger is necessary for audit and analysis.

The Android merge is an analysis task as well as a file transfer. Validate schema and token counts, confirm the adopted model/runtime/cache identities, distinguish platform memory measures, and avoid direct energy comparison when boundaries differ. Rerun the coverage audit, regenerate within-model plots and descriptive statistics, update the empirical narrative, and refresh every artifact from the same evidence snapshot. A changed phone setting or runtime becomes a labeled condition rather than silently entering an existing group.

Final submission checks remain separate. The report must have at least ten substantive pages plus references, cover all required sections, and include the working GitHub link. Slides must fit 15-20 minutes including the chosen demo plan; the short video must decode correctly and remain strictly below 120 seconds. Team rehearsal should cover quantization, bottlenecks, thermals, energy boundaries, quality limits, and failed runs. The deadline's year/time and whether Q&A counts in the slot require course verification. Publication, upload, and iCollege confirmation must be observed before those states are claimed.

References

[1] Qwen et al. (2025). Qwen2.5 Technical Report. arXiv:2412.15115v2. https://arxiv.org/abs/2412.15115v2

[2] Qwen. Qwen2.5-0.5B-Instruct-GGUF, official model repository. Artifact revision 9217f5db79a29953eb74d5343926648285ec7e67. https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF

[3] Qwen. Qwen2.5-1.5B-Instruct-GGUF, official model repository. Artifact revision 91cad51170dc346986eccefdc2dd33a9da36ead9. https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF

[4] ggml-org. llama.cpp source repository, pinned commit 79967ec596c0dacfd2251b085a57e79df292b1cc. https://github.com/ggml-org/llama.cpp/tree/79967ec596c0dacfd2251b085a57e79df292b1cc

[5] ggml-org. llama-bench documentation at the pinned revision. https://github.com/ggml-org/llama.cpp/blob/79967ec596c0dacfd2251b085a57e79df292b1cc/tools/llama-bench/README.md

[6] Merity, S., Xiong, C., Bradbury, J., and Socher, R. (2016). Pointer Sentinel Mixture Models. arXiv:1609.07843. WikiText dataset source. https://arxiv.org/abs/1609.07843

[7] ggml-org. Android build guidance at the pinned revision. https://github.com/ggml-org/llama.cpp/blob/79967ec596c0dacfd2251b085a57e79df292b1cc/docs/android.md

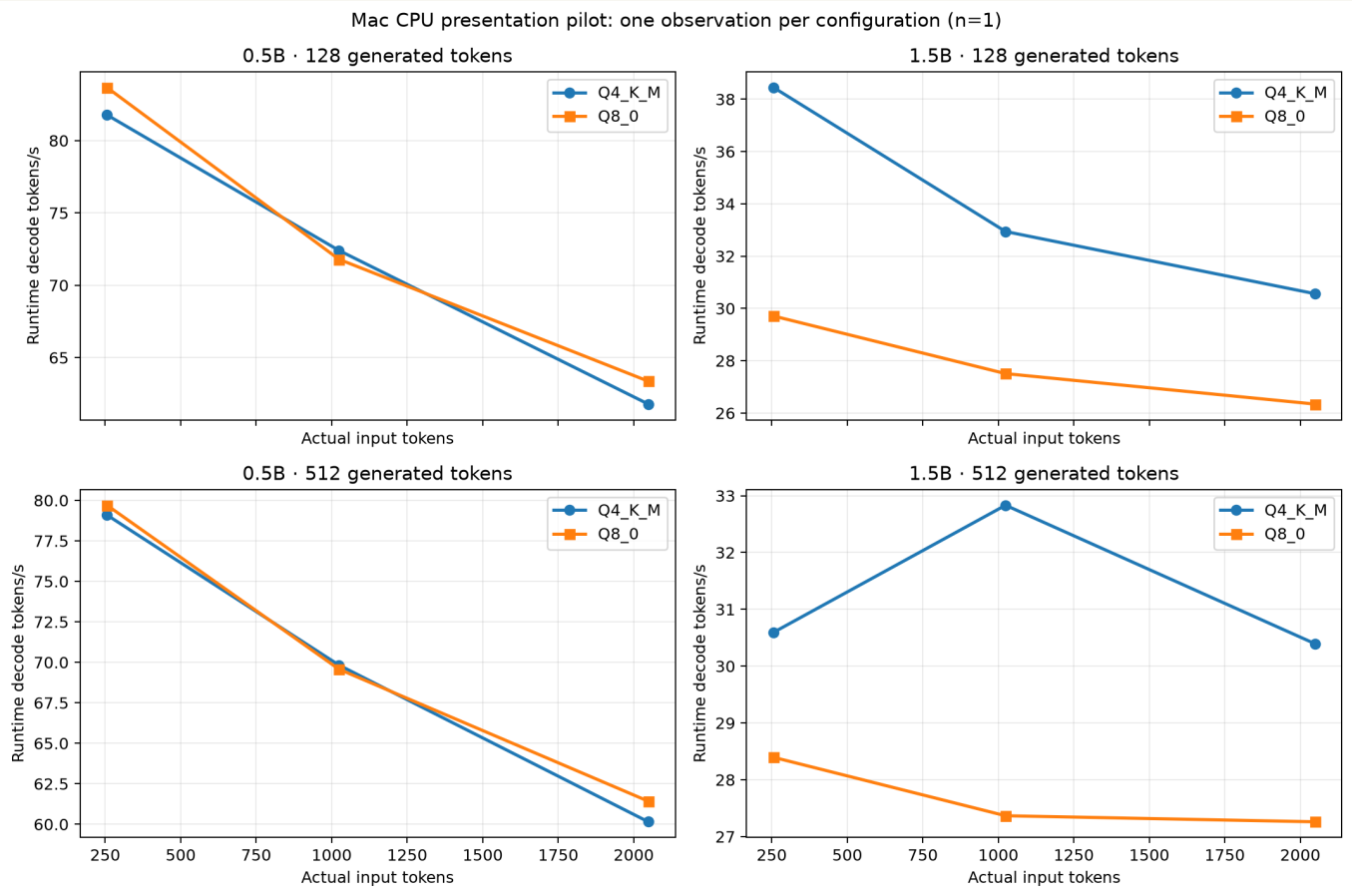
[8] Perfetto. Battery and power counters documentation. https://perfetto.dev/docs/data-sources/battery-counters

[9] Project team. Project objectives, historical proposal, model research, instructor requirements transcription, journal, and executable configurations. Repository: https://github.com/addiee13/project_effAI

[10] vladkens. macmon, Apple Silicon system monitor and JSON pipe output. https://github.com/vladkens/macmon. Accessed 2026-11-13. The project records local macmon version 0.11.0 and uses the tool's documented power-in-watts and temperature fields; these remain estimates from a private macOS API.

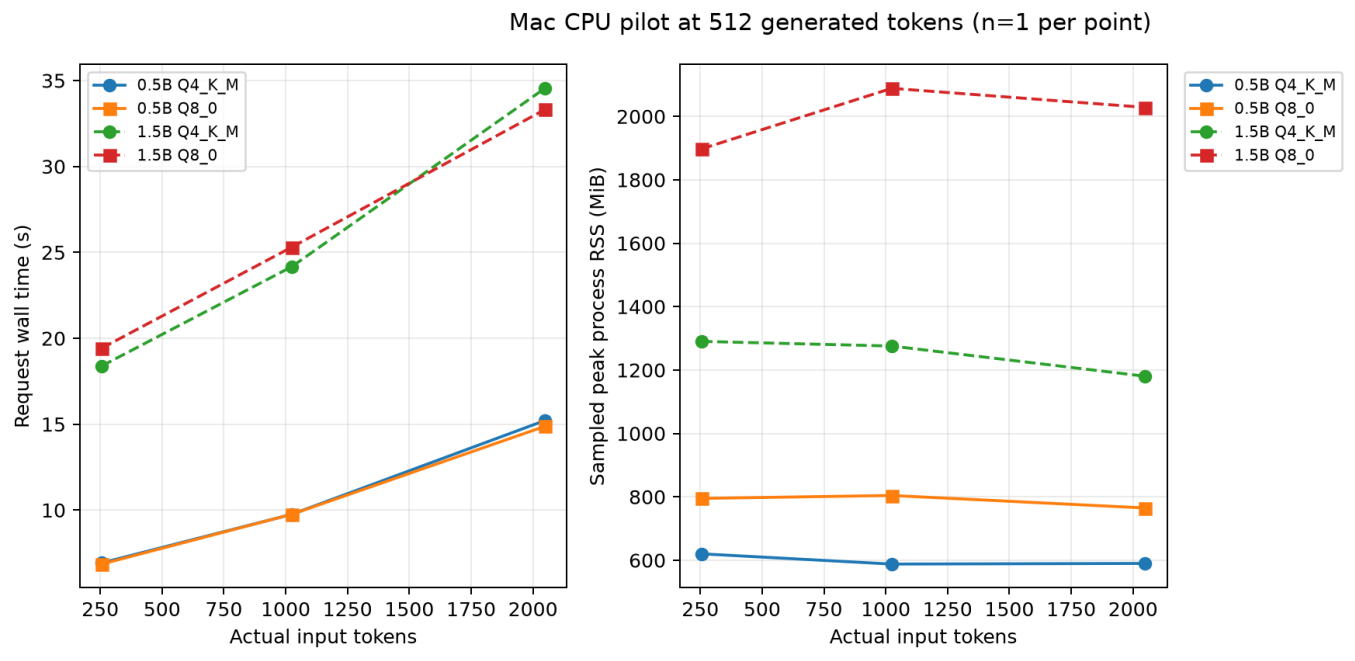
Primary model/runtime sources were checked on 13 November 2026. These references identify methods and artifacts, not independent evidence of local measured performance. The data snapshot and run identifiers embedded in the artifact validation manifest establish the report's local empirical provenance.

Runtime decode throughput by input length



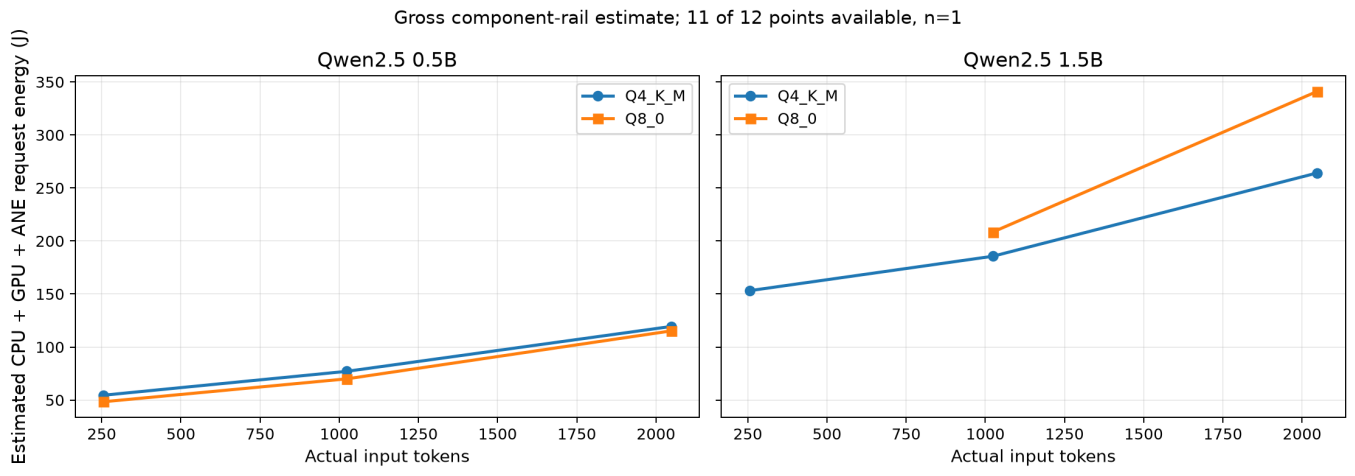
One observation per point (n=1). Runtime decode tokens/s is the runtime predicted token count divided by predicted_ms. Panels separate checkpoint and output budget; lines connect the three input lengths. Descriptive only; no error bars.

Request time and sampled process RSS at 512 generated tokens



One observation per point (n=1). Request wall time includes client request/stream delivery. RSS is sampled process RSS during each request, not lifetime maximum. Context capacity is fixed at 3,072; memory is not a linear KV-cache allocation curve.

Estimated component energy over each request



macmon CPU+GPU+ANE power estimates integrated over the client request interval. Eleven of twelve 512-token points have adequate sample brackets. Gross request energy, n=1; not whole-device energy and not isolated decode J/token.